

Document number: P2487R0
Date: 2021-11-12
Audience: SG21, EWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Attribute-like syntax for contract annotations

This paper is an attempt to structure the discussion on the choice of syntax for contract annotations.

[P0542] and [P2388] propose a syntax that appears similar to attributes:

```
int f(int i)
  [[pre: i >= 0]]
  [[post r: r >= 0]];
```

[P2461] proposes, similarly to [N1962], an alternate syntax, nothing like attributes:

```
int f(int i)
  pre{i >= 0}
  post(r){r >= 0};
```

And we could invent more syntax alternatives, for instance:

```
int f(int i)
  pre(i >= 0)
  post(r: r >= 0);
```

We try to identify clear criteria that would help make more objective decisions regarding the choice of syntax. We focus on the question: is attribute-like syntax suitable for contract annotations. We do not distinguish between different syntax choices that do not use the `[[_]]` notation. We believe that these criteria will be helpful even in the context outside of contract support in C++.

It is clear that contract annotations using attribute-like syntax proposed in [P0542] and [P2388] are not attributes according to the grammar productions of C++. Instead, the question we try to answer is: do we want the programmers to think of them as attributes?

"Ignorable"

Interestingly, the argument brought up in favour of the attribute-like syntax is that contract annotations are "ignorable"; whereas the arguments against the attribute-like syntax say that contract annotations are not "ignorable". This indicates that we do not have a shared understanding of what "ignorable" means.

We can get the best approximation from the C Standard ([N2596]), which defines what it means for a standard attribute to be ignorable.

6.7.11 p2:

Support for any of the standard attributes specified in this document is implementation-defined and optional. For an attribute token (including an attribute prefixed token) not specified in this document, the behavior is implementation-defined. Any attribute token that is not supported by the implementation is ignored.

4 p5:

A strictly conforming program shall use only those features of the language and library specified in this document. It shall not produce output dependent on any unspecified, undefined, or implementation-defined behavior, and shall not exceed any minimum implementation limit.

6.7.11.1 p3:

[...] A strictly conforming program using a standard attribute remains strictly conforming in the absence of that attribute. ¹⁶⁶⁾

Footnote 166:

Standard attributes specified by this document can be parsed but ignored by an implementation without changing the semantics of a correct program; the same is not true for attributes not specified by this document

The C++ Standard [N4892](#) has the following to say:

9.12.1 p1:

Attributes specify additional information for various source constructs such as types, variables, names, blocks, or translation units.

9.12.1 p6:

For an *attribute-token* [...] not specified in this document, the behavior is implementation-defined. Any *attribute-token* that is not recognized by the implementation is ignored.

The above definitions do not make it quite clear if "ignore" means "must parse correctly, but has no semantics", or "does not even have to parse correctly". For instance, is an implementation allowed to accept the following code without emitting any diagnostic?

```
[[noreturn(int)]] void f(auto i);
```

If "ignore" means "must parse correctly, but has no semantics" then this corresponds to the semantics of contract annotations in translation mode *No_Eval*, as described in [\[P2388\]](#).

For brevity, we call the property defined in C ("a strictly conforming program using a standard attribute remains strictly conforming in the absence of that attribute") *removable*. That is, we start from a hypothetical implementation that fully implements all standard attributes. We write a strictly conforming program that makes the use of a given attribute. We compile it using the implementation. We get a correct binary. Then we remove the attribute from the program. We expect the program to still remain strictly conforming and produce the same results.

We propose the following criterion for determining what feature is allowed (but not required) to be representable as an attribute in the program, and what it means for the implementation not to support a given attribute:

The standard describes for a standard attribute *A* (1) to what entity it is allowed to appertain, (2) what combination of tokens is allowed in *A* and (3) how it affects the translation of the program. An implementation is required to treat a program as ill-formed (produce at least one diagnostic) when (1) attribute *A* appertains to the entity that it is not allowed to appertain or (2) when the combination of tokens is not allowed by the specification of *A*. Under an implementation that supports *A*, the program, including the semantics of *A* must produce a well defined program. This is required regardless of whether the implementation chooses to support attribute *A* or not.

A well formed program using *A* is a program that (a) contains *A* and that (b) renders a well formed program under all implementations that support *A*.

An implementation may choose not to support attribute *A*. This means that a well formed program using *A*, the implementation may translate it as if all occurrences of *A* in the program were removed.

This definition makes standard attributes bad candidates for things that represent control structures, or generally things that might affect the output of the program. Whereas, it makes standard attributes good candidates for hints for generating warnings in tools, or for making the program ill-formed under some conditions. Standard attributes defined this way are also good candidates for optimization hints.

Under this definition, keyword `_Noreturn` in C is a good candidate for an attribute. So is `static_assert()` in C++, which is only used for emitting diagnostic messages. This indicates that there may be reasons for promoting a feature into a keyword even if it qualifies for an attribute, for instance to force all implementations to support the feature. This, however, hardly applies to contract annotations as proposed in [\[P2388\]](#), where we explicitly say that in translation mode *No_eval* an implementation is allowed only to check tokens between `[ [ and ] ]`, and we allow only one translation mode to be supported.

"Declarative" vs "imperative"

There are (at least) two possible models for describing semantics contract annotations. The first is that they are predicates (in the mathematical sense) for describing what constitutes an incorrect program at a given point in program execution. We can call it a "declarative" model. The other model is that contract annotations give a programmer a tool to execute arbitrary code at certain points in program execution, which can include control flows, such as `throw`-expression. We can call it an "imperative" model. An example of such model is proposed in [\[P1893\]](#), and it takes contract support close to `call` and `return` features described in [\[STROUSTRUP\]](#) and [\[DnE\]](#).

The preference of either of the models also affects the choice of the syntax for contract annotations. In the declarative model, the attribute-like syntax is a well suited candidate. But it is counterintuitive if we adopt the imperative model.

"Reorderable"

Although it is not specified anywhere formally, the intuitive expectation of the attributes is that while you can reorder two attributes inside a single *attribute-specifier* without changing the meaning, you cannot safely reorder two *attribute-specifiers*:

```
[[a, b]] int f(); // same as [[b, a]] int f();  
[[c]] [[d]] int g(); // different than [[d]] [[c]] int g();
```

This is compatible with contract annotations. One cannot reorder the following two contract annotations, as the short-circuiting property would be compromised:

```
int f(int * p)  
  [[pre: p != nullptr]]  
  [[pre: *p > 0]];
```

Ordering

Today, it is difficult to remember in which order declarators like `noexcept`, `const ->` and attributes should appear in function declarations after the parentheses:

```
int f(int) const noexcept [[gnu::fastcall]]; // correct or ill-formed?
```

Adding a new declarator to the list, looking similar to attributes, will introduce a new: sort of dilemma: how these contract annotations should be ordered relative to attributes appertaining to function type:

```
int f1(int i)                // correct declaration?
  [[pre: i > 0]]
  [[using gnu: fastcall]]
  [[post r: r > 0]];

int f2(int i)                // correct declaration?
  [[using gnu: fastcall]]
  [[pre: i > 0]]
  [[post r: r > 0]];

int f3(int i)                // correct declaration?
  [[pre: i > 0]]
  [[post r: r > 0]]
  [[using gnu: fastcall]];
```

Double square brackets as container

The present intuition that people might adopt is that two pairs of square brackets (an *attribute-specifier*) is a "container" that can hold zero, one or more comma-separated attributes:

```
[[[]]] int f(); // ok: zero attributes
[[noreturn]] int g(); // ok: one attribute
[[noreturn, nodiscard]] int h(); // ok: three attributes
```

Contract annotations would not fit into this intuition.

Type and Effect analysis

Imagine a system of annotations -- `[[writable]]`, `[[readable]]` -- that helps track the lifetime of a heap-allocated objects:

```
int* [[writable]] allocate ();
int* [[readable]] fill(int* [[writable]] p);
void read(int* [[readable]] p);
void deallocate(int* [[writable]] p);
```

Thus, function `deallocate()` requires the pointer to be `[[writable]]`. `[[readable]]` implies `[[writable]]`. Function `allocate()` produces a `[[writable]]` value. Function `fill()` upgrades the `[[writable]]` property to `[[readable]]`. This constitutes an *annotated type system* that can be subject to *Type and Effect analysis*. A tool can try to determine if `deallocate()` or `fill()` is ever called with a pointer that is not `[[writable]]`, or if `read()` is ever called with a pointer that is not `[[readable]]`.

We could invent a different system of annotations for tracking the ownership of a resource:

```
Res* [[in_session]] acquire ();
void use(Res* [[in_session]] r);
void release(Res* [[in_session, ends_session]] p);
```

This constitutes another Effect system, and a similar analysis could be performed using this system. And we could invent more and more such systems. The attributes are used to extend the type system with the effects system. The effects system is not enforced by the compiler, but is formal enough to enable a certain kind of analysis. The attribute syntax is a natural choice for expressing these effects.

Now, one of the use cases for contract support framework is to provide an automated way for describing the effect systems:

```
axiom writable(int*);
axiom readable(int*);

int* allocate() [[post r: writable(r)]];
int* fill(int* p) [[pre: writable(p)] [[post r: readable(r)]];
void read(int* p) [[pre: readable(p)]];
void deallocate(int* p) [[pre: writable(p)]];
```

With this capability, one can build a new kind of tool for performing Type and Effect analysis: one that is not tied to a fixed set of annotations, but can be taught to recognize arbitrary effect systems. Given this use case -- guiding type an effect static analysis -- the attribute-like syntax looks natural.

This use case is reflected by [api.express.unimplementable](#) and [dev.tooling](#) in [P1995].

Appertaining to function type

One argument against the attribute-like syntax, is that:

1. the position they are at means for attributes that they appertain to function type, and
2. they do not appertain to function type.

In order to validate this claim we would have to have a clear criterion for what qualifies as "appertaining to function type". The C++ or C Standards do not give us an answer. We could reach to non-standard attributes. One case highlighted by Aaron Ballman is a vendor-specific attribute `[[gnu::fastcall]]`:

```
void func(int i) [[gnu::fastcall]];

int main() {
    void (*fp)(int) = func; // error: type mismatch
}
```

See the Compiler Explorer example: <https://godbolt.org/z/3YzGb897f>. This illustrates that this specific attribute changes the type of the function it appertains to. But is it a general rule? Affecting the Effect system could be a natural generalisation of extending the type system.

One practical criterion for appertaining to function type is if we can use the same attribute to declare the function type rather than a function:

```
using AllocFun = void*(size_t s) [[pre: c > 0u]][[post r: r != nullptr]];

AllocFun quickAlloc, smartAlloc; // two functions with contract annotations?
```

Contract checking as UB

One way of looking at contract annotations is that they are "a sort of unspecified behavior" when any of them is violated, but where we actually control the behavior through compiler switches. They add nothing to the semantics of the program that does not violate the contract annotations. This seems to fit into the conceptual model of ignorable nature of the attributes.

Meta-annotations

Some use cases, like indicating that the cost of evaluating the contract predicate is greater than the cost of invoking the function, could naturally be expressed as attributes appertaining to contract annotations. However, this becomes impossible when contract annotations themselves look like attributes: you cannot have an attribute appertain to an attribute. This problem does not exist if a different syntax is used for contract annotations. The following, is an example taken from [\[P2461\]](#):

```
void store(P* p)
    pre{p != nullptr}
    [[audit]] pre{p->is_square()};
```

Detectability in reflection

It is possible that we might want in the future to inspect contract annotations through reflection. The question is: should things declared through attributes (or things pretending to be attributes) be detectable through reflection?

The usage of colon

Currently the syntax for *attribute-specifier* uses colon for separating the *attribute-using-prefix* from namespace-less attributes:

```
[[using clang: not_tail_called, optnone]] int f(int i);
```

Re-using colon in post-/pre-condition annotations would overload the meaning of the colon:

```
upsell greatest_negotiable_upsell(list const& l)
    [[post gnu: l.contains(gnu)]]
    [[using gnu: fastcall]];
```

The above example also illustrates that the presence of the colon alone cannot be used to easily tell apart contract annotations and *attribute-specifiers*.

Acknowledgements

Jens Maurer and Aaron Ballman offered substantial feedback on the intuition behind the attribute notation.

References

- [DnE] — Bjarne Stroustrup, "The Design and Evolution of C++", [Addison-Wesley, ISBN 0-201-54330-3](#).
- [STROUSTRUP] — Bjarne Stroustrup, "Wrapping C++ Member Function Calls" (<https://www.stroustrup.com/wrapper.pdf>).
- [N1962] — Lawrence Crowl, Thorsten Ottosen, "Proposal to add Contract Programming to C++ (revision 4)" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n1962.html>).
- [P0465] — Lisa Lippincott, "Procedural function interfaces" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0465r0.pdf>).
- [P0542] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P1893] — Andrew Tomazos, "Proposal of Contract Primitives" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1893r0.pdf>).
- [P2388] — Andrzej Krzemiński, Gašper Ažman, "Minimum Contract Support: either *No_eval* or *Eval_and_abort*" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2388r3.html>).

- [P2461] — Gašper Ažman, Caleb Sunstrum, Bronek Kozicki, "Closure-Based Syntax for Contracts" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2461r0.pdf>).